

AGREGACIONES

Las agregaciones en mongodb se utilizan para realizar operaciones de consulta más complejas, donde es posible agrupar documentos y aplicar funciones de agregación.

Para realizar operaciones de agregación mongo DB utiliza el **tubo o canal de agregación** esta contiene una o más etapas que procesan los documentos de entrada en una o más fases.

Cada etapa de la canalización de agregación realiza una operación en los documentos de entrada y devuelve los documentos de salida. Los documentos de salida pasan luego a la siguiente etapa. La etapa final devuelve el resultado calculado.



Las operaciones más comunes en cada etapa pueden ser una de las siguientes:

- ☐ **\$project** → seleccionar campos para los documentos de salida.
- ☐ **\$match** → seleccionar o filtrar los documentos a procesar.
- ☐ **\$limit** → limitar el número de documentos a pasar a la siguiente etapa.
- ☐ **\$skip** → omitir un número específico de documentos.
- ☐ **\$sort** → ordenar documentos.
- ☐ **\$group** → agrupar documentos por una clave específica.

Para realizar una agregación se utiliza el método **aggregate()**

Mongodb también incluye unas funciones de agregación, que permiten ejecutar operaciones con los grupos generados.

- ☐ **\$sum**: suma (o cuenta)
- ☐ **\$avg**: calcula la media
- ☐ **\$min**: mínimo de los valores
- ☐ **\$max**: máximo
- ☐ **\$push**: Mete en un array un valor determinado
- ☐ **\$addToSet**: Mete en un array los valores que se indiquen, pero solo una vez

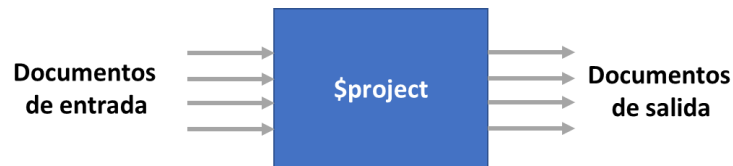
- ❑ **\$first:** obtiene el primer elemento del grupo, a menudo junto con sort
- ❑ **\$last:** obtiene el último elemento, a menudo junto con sort

Cada etapa de la tubería de agregación se puede usar de forma individual o continua.

Ejemplo 1

Mostrar solo los campos firstname, lastname, gender y jobtitle mediante una proyección.

Esta tubería de agregación tiene una sola etapa donde solo se definen los campos a mostrar antes de producir los documentos de salida.

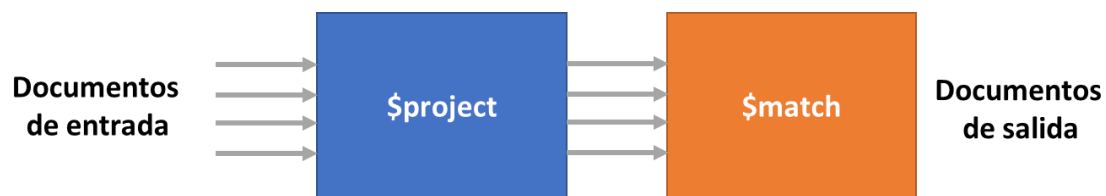


```
db.employee.aggregate([
  {
    $project:{
      firstname:1,
      lastname:1,
      gender:1,
      jobtitle: 1
    }
  }
])
```

Ejemplo 2

En la siguiente etapa del canal podríamos filtrar la data por cargo de empleado que corresponda a Auxiliar Estética

La tubería de agregación sería la siguiente:



Etapas 1 \$project: Se definen los campos a mostrar y se pasan los documentos a la etapa 2

Etapas 2 \$match: filtra los empleados por el campo jobtitle con el valor Auxiliar de estética, esta etapa pasará los documentos filtrados a la salida.

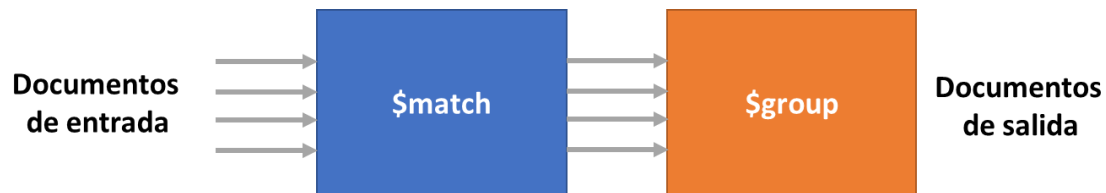
```

db.employee.aggregate([
  {
    $project:{
      firstname:1,
      lastname:1,
      gender:1,
      jobtitle: 1
    }
  },
  {
    $match:{
      jobtitle:"Auxiliar de estética"
    }
  }
])

```

Ejemplo 3

Obtener la cantidad de Auxiliares de estética agrupados por genero



Esta tubería de agregación tiene también dos etapas:

Etapla 1: \$match filtra los empleados por el campo jobtitle y el valor Auxiliar de estética y pasa los documentos filtrados a la etapa 2 \$group.

Etapla 2: \$group agrupa los documentos filtrados por género mediante el campo gender y utiliza la función agregada \$sum para calcular la cantidad de cada uno. \$group crea una nueva colección de documentos donde cada documento contiene dos campos _id y cantidadempleados que se pasa a la salida

```

db.employee.aggregate([
  {
    $match:{jobtitle:'Auxiliar de estética'}
  },
  {
    $group:{
      _id:"$gender",
      "cantidad empleados": { $sum: 1 }
    }
  }
])

```

Ejercicios de práctica

Para cada uno de los enunciados propuestos indique

- ☐ ¿Cuál sería la tubería de agregación para la siguiente consulta propuesta?
- ☐ ¿Qué sucede en cada etapa de la tubería de agregación?
- ☐ ¿Cómo quedaría la agregación final?

1. Obtener la cantidad de empleados de la veterinaria agrupados por género y ordenarlos de menor a mayor.
2. Identificar cuantos auxiliares de estética hay por género y organizarlos de forma ascendente por el género.
3. Calcular el total de la nómina de los empleados por cargo y ordenar alfabéticamente por cargo.
4. Calcular el total de la nómina de los empleados por cargo y mostrar solo los que superen los 5 millones de pesos.

Reto de aplicación

Cree en su servidor de mongo db la base de datos **runningapp** y en ella importe la colección adjunta **corredores.json**.

En un documento de Word, denominado **Taller_Agregaciones_Nombre_Apellido**, realice para cada enunciado propuesto lo siguiente:

- ☐ Esquema gráfico de la tubería de agregación que se aplicaría
 - ☐ Código de la agregación
 - ☐ Pantallazo de resultado de la agregación realizada que permita ver los documentos solicitados en el enunciado.
1. Identificar total de minutos y total de kilómetros recorridos por cada usuario (*investigue cómo concatenar nombre y apellido*) de la aplicación running
 2. Obtener el promedio de minutos mensuales realizados por cada corredor
 3. Obtener el promedio de kilómetros mensuales de cada corredor, siempre y cuando ése promedio sea mayor a 5K
 4. Obtener el total de kilómetros recorridos agrupados por corredor y por mes (*investigar cómo se puede agrupar por dos campos*), ordenar por mes
 5. Ver por cada corredor las distancias recorridas (*investigar uso de operador \$addToSet*)
 6. Determinar cuántos corredores están interesados en cada afición (*investigar uso \$unwind*)
 7. Complete 3 agregaciones más utilizando las etapas y funciones de agregación que usted necesite.